ACN PROJECT · TEAM 7

Frame-aware multipath QUIC with a learned bandwidth predictor

Streaming live lectures over two mobile connections without the video freezing

On real cellular networks, standard schedulers deliver **16.4%–18.2%** of video frames too late. Ours: **10.6%–13.5%**.

Kabir Bajaj and Team 7 · 29 October 2026

The setting

- Students watch live lectures on a **phone with mobile data**
- Many phones have **two SIM cards**: two connections, each slow and unsteady (a few Mbit/s, 50–150 ms delay)
- **Multipath QUIC** uses both at once; a **scheduler** picks the path for every packet



I-frame: full picture, ~55 KB, every 2 s P-frame: only the changes, ~8 KB, 24 per second

Frame i, produced at time s_i, must arrive completely by $s_i + D$, with playback delay D **= 300 ms**. If not, it is a **deadline miss** and the video freezes.

The problem: schedulers see bytes, not frames

- **MinRTT** (TQUIC's default): next packet on the lowest-delay path with room
- **Round-robin**: alternate between paths
- **ECF** (earliest completion first): wait for the fast path if that finishes sooner
- **Redundant**: every packet on both paths (half the capacity spent on copies)
- None knows where a frame starts or ends, or when it is due: a **split frame waits for its slower half**, and **small frames get stuck behind big ones**

EXAMPLE: ONE I-FRAME ON A SLOW PATH

$$\frac{45\ \text{KB} \times 8}{1.5\ \text{Mbit/s}} + 30\ \text{ms} = \mathbf{270\ ms}$$

That is 270 of the 300 ms budget, before counting frames already queued.

Research questions: (1) how often do standard schedulers miss deadlines? (2) can a scheduler that knows each frame's size and deadline, with a learned speed prediction, do better? (3) which part of the design matters?

Our scheduler: predict when each frame will arrive

$$T_p = \frac{Q_p + S}{B_p} + d_p$$

T_p arrival time on path p · S frame size · Q_p bytes waiting ahead of it · B_p path speed · d_p = sRTT$_p$/2 one-way delay

RULE 1: WHOLE FRAME IF A PATH MAKES IT

$$p^* = \operatorname{argmin}_p T_p$$

$$\text{use } p^* \text{ if } T_{p^*} \le h \cdot (\text{deadline} - \text{now})$$

Then all S bytes go on p^*. Safety margin h = 0.85 for the hand-tuned version, h = 1 for the learned one (its prediction is already cautious).

RULE 2: OTHERWISE SPLIT, BOTH PARTS FINISH TOGETHER

$$T^* = \frac{S + Q_A + Q_B + B_A d_A + B_B d_B}{B_A + B_B}$$

$$s_p = B_p(T^* - d_p) - Q_p$$

$s_A + s_B = S$, and both parts arrive at the same time T^*.

Q_p = bytes of earlier frames planned for p + max(0, in-flight$_p$ – B_p·minRTT$_p$). Each path sends only its own frames, so small frames overtake big ones. A frame already past its deadline may use any path.

The ML part: predicting the speed B_p

- If B_p is **too high**, rule 1 trusts a path that will be late. BBR's own estimate is too high **38.2%** of the time on our test recordings.
- **Input** x: 44 numbers per path from the last second (delivery rate, send rate, queueing delay, loss, cwnd, BBR's estimate, …)
- **Target**: $y = \ln(1 + C/1000)$, C = true average speed over the next 0.5 s (bytes/s)
- **Data**: 6 recorded cellular networks for training, 2 others for testing

WHAT THE SCHEDULER USES

$$B_p = 1000\,(e^{\hat{y}_{0.1}} - 1) \;\Rightarrow\; P(C < B_p) \approx 10\%$$

SMALL NEURAL NETWORK (MLP): 44 → 32 → 32 → 4

$$h_1 = \mathrm{ReLU}(W_1 x + b_1)$$
$$h_2 = \mathrm{ReLU}(W_2 h_1 + b_2)$$
$$\hat{y} = W_3 h_2 + b_3$$

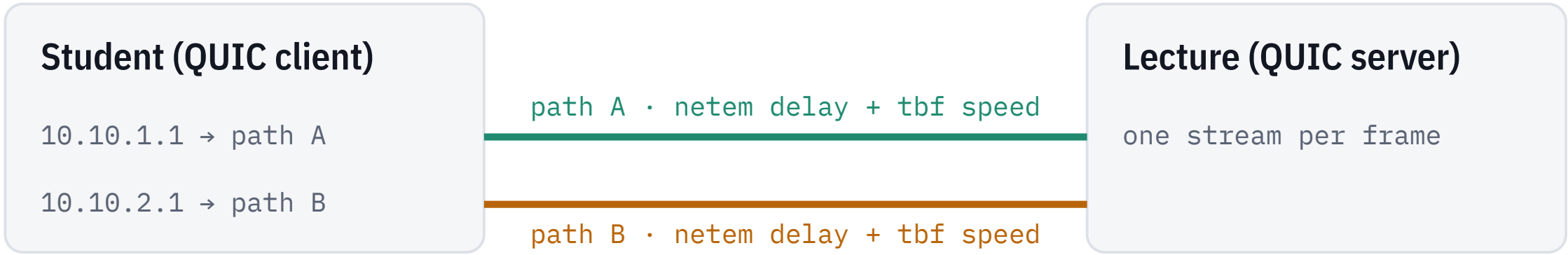
Four outputs $\hat{y}_\tau$: the 10th, 20th, 30th and 50th percentile (τ = 0.1, 0.2, 0.3, 0.5).

PINBALL LOSS

$$L_\tau(y, \hat{y}) = \max(\tau(y - \hat{y}), (\tau - 1)(y - \hat{y}))$$

For τ = 0.1, too high costs 0.9, too low 0.1: **9× penalty for overestimates**.

How we test: real QUIC code on an emulated network



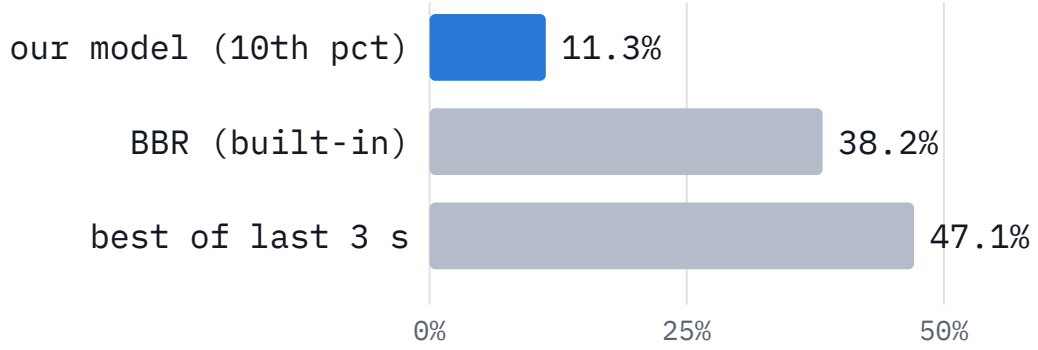
- One laptop: Linux **network namespaces** joined by two virtual links (**veth**)
- Linux **tc**: netem adds delay, tbf sets speed with a 100 ms queue; every 100 ms the speed is replayed from **real cellular recordings**
- A real 60 s lecture (1,441 frames, 1.57 Mbit/s); 8 schedulers × 5 scenarios × 5 runs = **200 runs**, same network for the same run number

SCENARIO	PATH A	PATH B
symmetric	4 Mbit/s, 40 ms	4 Mbit/s, 40 ms
rtt_asym	4 Mbit/s, 30 ms	4 Mbit/s, 150 ms
low_bw	1.5 Mbit/s, 60 ms	1 Mbit/s, 120 ms
trace	AT&T LTE	T-Mobile 3G
outage	4 Mbit/s	off 2 s / 30 s

Result 1: the model predicts speed better

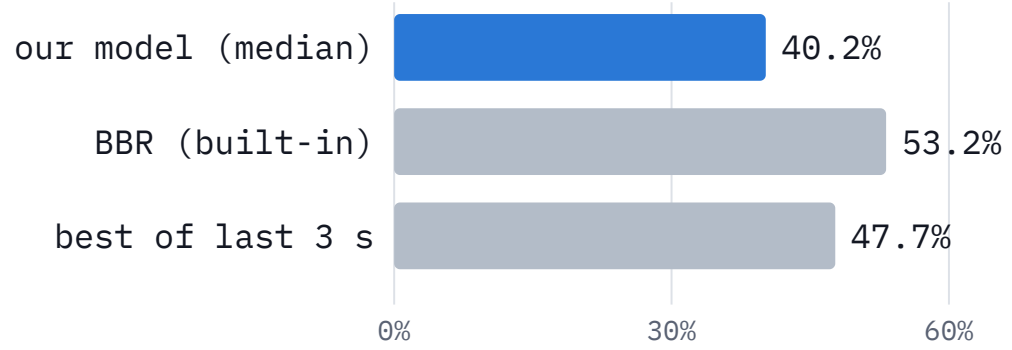
How often the guess is too high

$$\mathrm{P}(\hat{B} > C)$$



Typical error

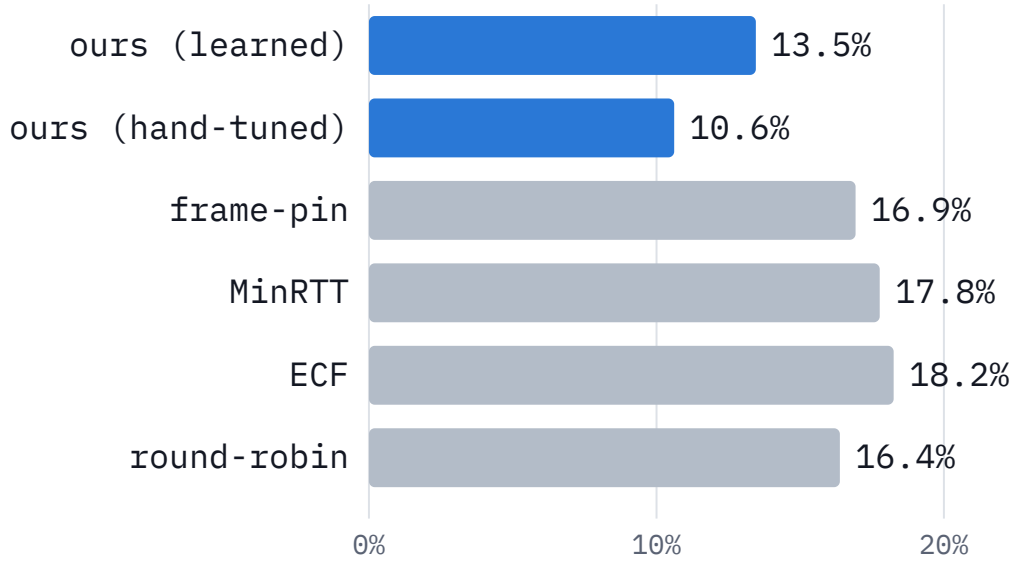
$$\mathrm{median}\ |\hat{B} - C| \ / \ C$$



$\hat{B}$ = estimate, C = true speed over the next 0.5 s, at 31,912 moments on the 2 recordings the model never saw. Blue = our model. **Shorter bars are better.**

The 10th percentile is too high **11.3%** of the time, close to the 10% it promises (**calibrated**), against **38.2%** for BBR.

Result 2: fewer late frames on real cellular networks



Trace scenario, median of 5 runs. Blue = ours. **Shorter is better.** Redundant 52.8% and single path 72.9% are off the scale.

$$\text{miss rate} = \frac{1}{N} \Sigma_i \, 1[c_i > s_i + D]$$

c_i = when frame i is complete, N = 1,441 frames, D = 300 ms

- Late frames: **10.6%–13.5%** with ours vs **16.4%–18.2%** standard
- Run by run, learned beats MinRTT and ECF in 3 and 3 of 5 runs; freezes less in 4 and 4 of 5 (1.3 s vs 1.6–1.8 s per minute)
- **Ablation**: frame-pin keeps frames whole but has no deadline check or split: 16.9%, no better than standard

Conclusions

WHAT WE FOUND

- Planning frames against deadlines cuts late frames from **16.4%–18.2%** to **10.6%–13.5%**
- Our ML model is the **most accurate** speed estimate
- But it is too cautious: the hand-tuned version is still a bit better (10.6% vs 13.5%; learned better in 2 of 5 runs)

LIMITATIONS

- Emulated network, not a live one
- Recordings from US networks
- One 60 s lecture, 5 runs each

NEXT STEPS

- Choose τ from the predicted spread: **less caution** on steady links
- Retrain on the learned scheduler's own traffic
- More recordings, longer lectures

Thank you · Questions?